

Graph Theory Project

Finding shortest paths using the Floyd–Warshall algorithm

SM601I - Graph theory

Year 2025/2026 – Department of Mathematics

IMPORTANT: Take the time to carefully read this document. It contains important instructions that you must follow. Any failure to do so will affect your grade.

The Floyd–Warshall algorithm makes it possible to compute minimum-value paths from any vertex to any other vertex in a directed weighted graph. This algorithm is based on Warshall’s algorithm, which is dedicated to computing the transitive closure of a graph. The computation of this transitive closure is adapted in order to retain, among all paths connecting two vertices, those with the smallest value.

The Floyd–Warshall algorithm is not covered in the lectures of this course. This project therefore includes a small research component on your part.

Work to be done

The project contains two parts.

1. IMPLEMENTATION OF THE FLOYD–WARSHALL ALGORITHM

Main loop: Your program must be able to execute the required processing on a sequence of graphs chosen by the user. During the oral examination, it must not be necessary to restart your program to test it with several graphs.

Processing a graph:

- (1) First, the user specifies the graph to analyze. Graphs will be identified by numbers.
- (2) The graph is then loaded into memory from its description contained in a text file. **See appendix.**
- (3) The graph that is read is stored in data structures. From this point on, your program **must never** access the file again: only the in-memory representation is used.
- (4) The graph is then displayed on the screen in matrix form: a matrix of values. Pay careful attention to readability, especially column alignment and vertex identification (row and column headers).
- (5) The Floyd–Warshall algorithm is executed. The intermediate values of the algorithm’s data (matrices usually called ‘L’ and ‘P’) are displayed.
- (6) After reading the result of the algorithm, your program indicates whether the graph contains at least one absorbing circuit.

- (7) If no absorbing circuit is present, the final processing consists in displaying the minimum-value paths. You may display them all, or implement a user interface loop:

```
Path?
If yes, then
Starting vertex? Ending vertex?
Display the path
Restart?
If no, then stop
```

2. CONSTRUCTION OF AN APPLICATION EXAMPLE

You are asked to construct a real example that requires determining minimum-value paths for all pairs of vertices in a graph. You must therefore

- describe the context of the problem,
- describe the data (which must be sufficiently large),
- run your algorithm on these data to find the solutions,
- interpret the obtained solutions.

Graphs to consider

Your program must correctly handle any graph that satisfies the following criteria:

- Directed graph
- Weighted graph — integer numerical values (negative values allowed, '0' allowed).
- Vertices represented by integers from '0' to 'number of vertices minus 1'.
- At most one arc from a vertex x to a vertex y .

Your program must not impose any limit on the number of vertices, arc values, or number of arcs.

Test graphs

Test graphs will be provided in graphical form on Monday, March 9, which gives you enough time to encode them as .txt files in the format you have chosen. These files must be prepared in advance and will be part of the submission. They must be available on your computer at the time of the oral defense.

Do not wait for these graphs to test your program! It would then be too late. During development, feel free to use all graphs studied in lectures and tutorials, and many others. The more tests you perform, the more confident you can be that your program works correctly.

Work organization

Team work:

- there must be at most 8 teams per TD group.
- each team must have at least three students.
- each team must have at most five students.
- **Deadline for the constitution of project groups: Sunday, March 1 at 23:59.** The delegates must send the list of groups to the TD instructors.
- Possible issues: If there is a problem within a team (a student not contributing, illness, etc.), you must inform your instructor immediately. No team change will be accepted without direct approval from the instructor.

Programming language:

Your choice: C, C++, Python, Java.

However, the chosen language must be sufficiently mastered by all team members so that everyone can participate. During the oral defense, the instructor may ask any question to any team member.

Submission:

Submission deadline (via Moodle): Sunday, March 22 at 23:59.

Submission must include:

- **SOURCE CODE:** All source files written by your team, excluding any automatically generated files, and properly commented.
- All **.TXT FILES** of the test graphs, in the same directory as the code.
- **EXECUTION TRACES** in a **.txt** file. You must execute your program on all test graphs and provide the corresponding execution traces.
- A **PDF REPORT** (maximum five pages) explaining the real example, the data, and the results.

Oral examination:

- 30 minutes per team: five minutes at the beginning to run your program, followed by 25 minutes of questions and answers. No need to prepare any slides.
- **The oral examination grade is collective.** Your instructor may ask any question to any member of the team, whether or not they participated in developing the relevant part of the program. All team members are therefore responsible for ensuring that all other members have the same level of understanding of the project.
- The oral examinations have a strict schedule. You must be ready on time, which implies:
 - waiting outside the room and entering when it is your turn;
 - having your computer ready with all programs and graph files;
 - ensuring your battery is fully charged;
 - having the computer started and in sleep mode;
 - having a backup computer ready if needed.

Appendix – Example of a file representing a graph

Important: This appendix contains only an example of file structure. You may choose your own structure provided that:

- the file structure is simple and easy to understand,
- the graph can be easily modified directly in the file.

Your file may, for example, have the following structure:

Line 1 – Number of vertices

Line 2 – Number of arcs

Lines 3 to 3 + number of arcs – Initial vertex, terminal vertex, arc value

With this file model, if it contains the following text:

```
4
5
3 1 25
1 0 12
2 0 -25
0 1 0
2 1 7
```

this corresponds to a graph containing 4 vertices numbered from 0 to 3, and 5 arcs $(3, 1)$, $(1, 0)$, $(2, 0)$, $(0, 1)$, $(2, 1)$ with respective values 25, 12, -25 , 0 and 7.